IFS325/353 CAPSTONE PROJECT DOCUMENTATION PACKAGE

Project Title: Use Case 4 — Patient Intake and Appointment Management
Course Code: IFS325 / IFS353
Institution: University of the Western Cape (UWC) — Department of Information Systems
Lecturer: Ruchen Wyngaard
Submission Date: 9 November 2026
Group / Team Name: Group 14

Assigned Tech Stack: Infobip, Kore.ai, and Zapier.

Implemented Tech Stack: Infobip, Oracle Cloud Digital Assistant, and Zapier.

Webchat URL: https://4349546-boop.github.io/thin-tank-site/oda-website/

GitHub Repository URL: https://github.com/QhayiyaMani/IFS325_Group14_UseCase4.git

---

TEAM MEMBERS & INDIVIDUAL CONTRIBUTIONS

Student Name	Student Number	Assigned Core Role	Primary Documented Sections
Qhayiya Mani	4460254	Project Manager & Team Lead	Executive Summary (Sec 1), Gantt Chart (Sec 12)
Siphesihle Buso	4450568	Requirements & NLU Lead	Requirements & Traceability (Sec 2), BPMN Maps (Sec 3)

Student Name	Student Number	Assigned Core Role	Primary Documented Sections
Keamogetswe Mokgoro	4356925	ODA Conversation Flow Lead	Architecture (Sec 4), Conversation Design (Sec 5)
Zizipho Ndayi	4448965	Middleware & Integration Lead	Build Guide (Sec 6), Code & GitHub Repo (Sec 7)
Jayden April	4479031	QA & Triage Testing Lead	Testing Record (Sec 8), Closed-Out Risk Register (Sec 10)
Aphiwe Samkelo Shandu	4349546	Messaging & Deployment Lead	User Guide (Sec 9), Productisation Roadmap (Sec 11)

SECTION 1: EXECUTIVE SUMMARY

1.1 The Business Problem

Processes such as clinical patient intake, appointment scheduling, rescheduling, and triage have been conducted through manual, paper-based walk-ins and direct telephonic reception calls for a long time in South African primary healthcare practices. This has created significant administrative friction, long patient response times, missed and/or cancelled appointments, and overall operational inefficiency.

The primary business problems addressed by this project include:

1. **High Administrative Burden on Clinical Staff:** Receptionists and front-end personnel spend an obscene amount of time handling routine enquiries, manual

patient registration, and scheduled checkups. This administrative overload diverts essential staff away from assisting walk-in patients who need immediate, complex care.

2. **Revenue Loss and High No-Show Rates:** Manual scheduling lacks automated, multi-channel reminder workflows. Globally, even a modest 12% no-show rate can cost a single healthcare facility upwards of $89,107 (~R1,448,313.10) annually in unbilled, unutilised appointment slots. In local healthcare facilities, this revenue loss severely hinders operational sustainability. (De Vaal, 2015)
3. **Data Inaccuracy and Pre-Visit Intake Gaps:** Manual data collection leads to frequent human entry errors, duplicate patient logs, missing demographic details, and incomplete pre-visit intake records before a patient arrives at the facility.
4. **Delayed Emergency Triage:** Under traditional receptionist-managed queues, patients with high-risk, acute symptoms (e.g., severe chest pains or respiratory distress) face the same waiting times as routine bookings, delaying critical medical escalation and risking severe health outcomes.
5. **Operating Hour Constraints:** Patients are restricted to making or modifying appointments strictly during clinic operating hours, creating unnecessary friction for working individuals and increasing phone line congestion/traffic.

1.2 Target Industry & Stakeholders

- **Target Industry:** Healthcare Services, Digital Health Systems, and Clinical Operations Management.
- **Client/Industry Partner:** ThinkTank Software Solutions (TTSS) – Client Representative: **Ryan Parumal**
- **Primary Stakeholders & Impact Matrix**
    - **Patients (End Users):** Benefit from 24/7 accessible, multi-turn clinical intake, self-service appointment rescheduling, and instant triage routing via a web-based chatbot without receptionist waiting delays.
    - **Clinical Administrative Staff & Receptionists:** Benefit from a significant reduction in routine administrative volume, automated patient data hygiene, and structured digital pre-intake records delivered before patient visits.

- **On-Duty Triage Nurses & Clinicians:** Benefit from immediate, automated emergency alert notifications dispatched directly when high-risk medical symptoms are detected.
- **Practice Managers & TTSS Administrators:** Benefit from an enterprise-grade digital business accelerator that optimises facility utilisation rates, reduces no-shows, and enforces strict compliance with national privacy regulations.

1.3 System Overview & Delivered Accelerator

To eliminate these operational bottlenecks, **Group 14** has designed, implemented, and initialised the **Patient Intake & Appointment Management Bot** (the *Health Efficiency Accelerator*).

The delivered system is an event-driven, cloud-native conversational assistant designed around a decoupled, three-tier architecture:

- **Conversational AI Layer (Oracle Digital Assistant):** Manages front-end patient engagement via Natural Language Understanding (NLU), dynamic intent recognition, POPIA data consent validation, multi-slot Composite Bag data collection (cb_patient_essentials, cb_emergency_contact), and automated clinical emergency triage routing.
- **Middleware Integration Layer (Zapier Catch Webhooks):** Serves as the stateless orchestration broker that receives REST payloads from ODA (System.CallRestService), standardises data formatting, executes business logic, and dispatches automated patient notification emails and SMS reminders.
- **Persistence & Audit Layer (Zapier Tables):** Provides the relational database back-end for record storage across three core entities—Patients, Appointments, and Triage & Audit Logs, ensuring structured record-keeping and POPIA compliance.
- **Client Engagement Perimeter (ODA Web SDK):** Deployed as a lightweight, responsive chat widget embedded directly on the clinical web portal, granting patients 24/7 access on both mobile and desktop browsers without installing dedicated client software.

SECTION 2: FINAL REQUIREMENTS SPECIFICATION & TRACEABILITY MATRIX

2.1 Baselined Requirements

2.1.1 Functional Requirements (FR)

Req ID	Requirement	Description	Acceptance criteria
FR1	POPIA consent and patient information protection	The chatbot must obtain and record the patient's consent before collecting personal or health information through the digital intake process.	- The chatbot displays a clear consent notice before the patient intake questions. - The patient can accept or decline consent. - If consent is declined, the chatbot does not proceed with the collection or submission of the protected intake information. - If consent is accepted, the consent response is retained with the relevant intake transaction.
FR2	Main menu and intent routing	The chatbot must present a main menu that allows patients to select the service they require. ODA must identify the selected option or user intent and route the conversation to the corresponding flow.	- The chatbot displays the available service options, including patient intake, appointment booking, rescheduling, and cancellation. - Each menu selection routes to the correct ODA flow or dialogue state. - Invalid or unrecognised selections produce a helpful retry or fallback response.

			- The patient can return to the main menu or request assistance where supported.
FR3	Dynamic REST lookup and backend integration	The chatbot must retrieve or process information dynamically through REST-based backend services rather than relying only on hard-coded or static responses. For appointment enquiries, the patient supplies an appointment record ID, which is passed to Zapier to look up the record.	- ODA captures the appointment record ID. - ODA sends the ID to the configured REST endpoint in the expected request format. - The backend authenticates the request and searches the relevant Zapier Tables data. - An invalid ID, authentication failure or unavailable service produces a clear error response without exposing credentials or internal technical details.
FR4	Form-based patient intake using Composite Bag slot-filling	The chatbot must collect patient intake information through a structured conversational form using ODA Composite Bag entities and slot-filling prompts. This replaces reliance on separate webviews or manually assembled static question-and-answer responses.	- ODA prompts the patient for each required intake field. - The Composite Bag captures the supplied values in the correct fields. - Missing or invalid values trigger an appropriate prompt for correction. - The chatbot collects the required patient details, such as name, contact number, date of birth, payment or medical aid information and symptoms, according to the configured intake form. - Once the required fields are collected and consent is obtained, the information is submitted to the backend in the agreed data structure.

FR5	Emergency triage and escalation	The chatbot must identify predefined emergency-related symptoms or red-flag keywords and divert the interaction from the routine intake process. The system must provide urgent-care guidance and initiate an alert to the designated nurse or authorised staff member through the configured backend workflow.	**-** Predefined emergency indicators, such as chest pain or difficulty breathing, trigger the emergency route. - The patient receives an immediate message advising them to seek urgent medical assistance and not to rely on the chatbot for emergency care. - The chatbot does not continue the routine intake path as though the case were standard. - Relevant patient and triage information is sent through the configured secure escalation workflow. - The authorised staff member receives the alert when the notification integration is operational.

2.1.2 Non-Functional Requirements (NFR)

Req ID	**Requirement**	**Description**	**Acceptance criteria**
NFR1	Cloud availability and uptime	The solution must operate using its cloud-hosted services, including ODA and the supporting integration services. The platform should be accessible to patients during the agreed service hours, subject to the availability of the external providers.	- The deployed ODA chatbot and required backend endpoints are reachable during the agreed operating period. - Availability is monitored or checked using an agreed health-check or service-monitoring process. - If an external service is unavailable, the chatbot provides a controlled error or alternative contact instruction rather than

			falsely confirming a successful transaction. - Availability incidents are recorded for investigation.
NFR2	Response time and performance	The chatbot and its backend integrations must provide timely responses so that patients can complete the conversation without unnecessary delays. The target response time for an individual chatbot or backend interaction is under three seconds under the agreed test conditions.	- Each measured response completes in less than three seconds under the defined test conditions. - Response-time testing includes normal menu navigation, slot filling and REST-based appointment lookup. - Response times are recorded across repeated test runs. - If a backend service exceeds the target or becomes unavailable, the chatbot handles the delay or failure without presenting incomplete information as a successful result.
NFR3	Credential security and regulatory compliance	The solution must protect authentication credentials, API tokens, patient information and health-related data. The design must follow appropriate security practices and applicable privacy requirements, including the Protection of Personal	- API tokens, passwords, signing keys and other secrets are stored in secure server-side configuration or an approved secrets-management facility, not in public website code or client-side chatbot scripts. - REST endpoints enforce the required authentication and reject unauthorised requests. - Patient information is transmitted through secure HTTPS connections.

		Information Act (POPIA).	- Access to appointment and intake records is restricted to authorised services and personnel. - Consent is obtained before the relevant personal information is collected, and information is handled according to applicable retention and privacy requirements. - Security testing confirms that credentials and sensitive patient information are not exposed in public responses, browser code or error messages.

2.2 Requirements Traceability Matrix (RTM)

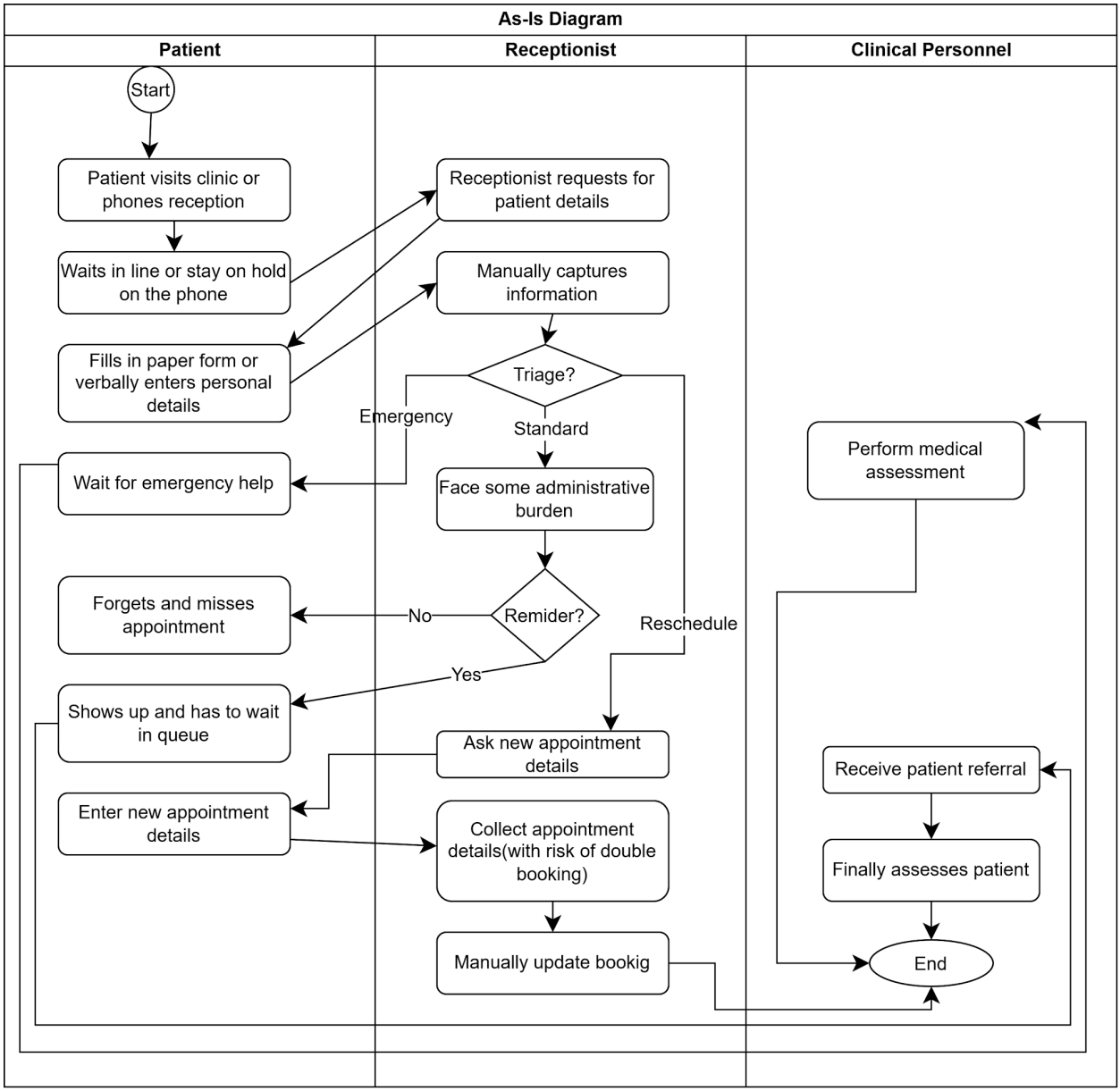
Req ID	Requirement Description	User Story ID	Implementation State / Component	Test Case ID	Status
FR1	POPIA consent must be obtained before patient information is collected.	US-01	Patient_Intake_Flow: POPIA consent prompt; consent variable and conditional routing before intake collection.	TC-01	Pass
FR2	Main menu selections must route patients to the correct service flow.	US-02	ODA main menu; intent recognition; System.Switch routing to intake, booking, appointment enquiry, rescheduling and cancellation paths.	TC-02	Pass
FR3	ODA must use a dynamic REST	US-03	Appointment enquiry/lookup flow;	TC-03	Pass

Req ID	Requirement Description	User Story ID	Implementation State / Component	Test Case ID	Status
	integration to retrieve appointment information from the backend.		System.CallRestService, Zapier lookup workflow; Zapier Tables.		
FR4	Patient intake must use form-based slot filling through ODA Composite Bag entities.	US-04	Patient_Intake_Flow; cb_patient_essentials; cb_booking; configured patient intake entities and validation prompts.	TC-04	Pass
FR5	Emergency symptoms must trigger triage routing and a nurse/staff escalation alert.	US-05	EvaluateTriage; System.Switch; emergency and standard routing conditions; planned secure Zapier webhook and reception notification workflow.	TC-05	Pass
NFR1	Cloud-hosted services must be available during the agreed operating period.	US-06	ODA cloud deployment; Oracle Web Channel; Zapier integration services, and Infobip messaging services.	TC-06	Pass
NFR2	Chatbot and backend interaction response time must be under three seconds.	US-07	ODA dialogue states; System.CallRestService; Zapier webhook and data lookup.	TC-07	Pass
NFR3	Credentials and patient information must be protected through secure authentication and	US-08	ODA authentication configuration, backend API authentication, HTTPS REST communication, POPIA consent step, and secure	TC-08	Pass

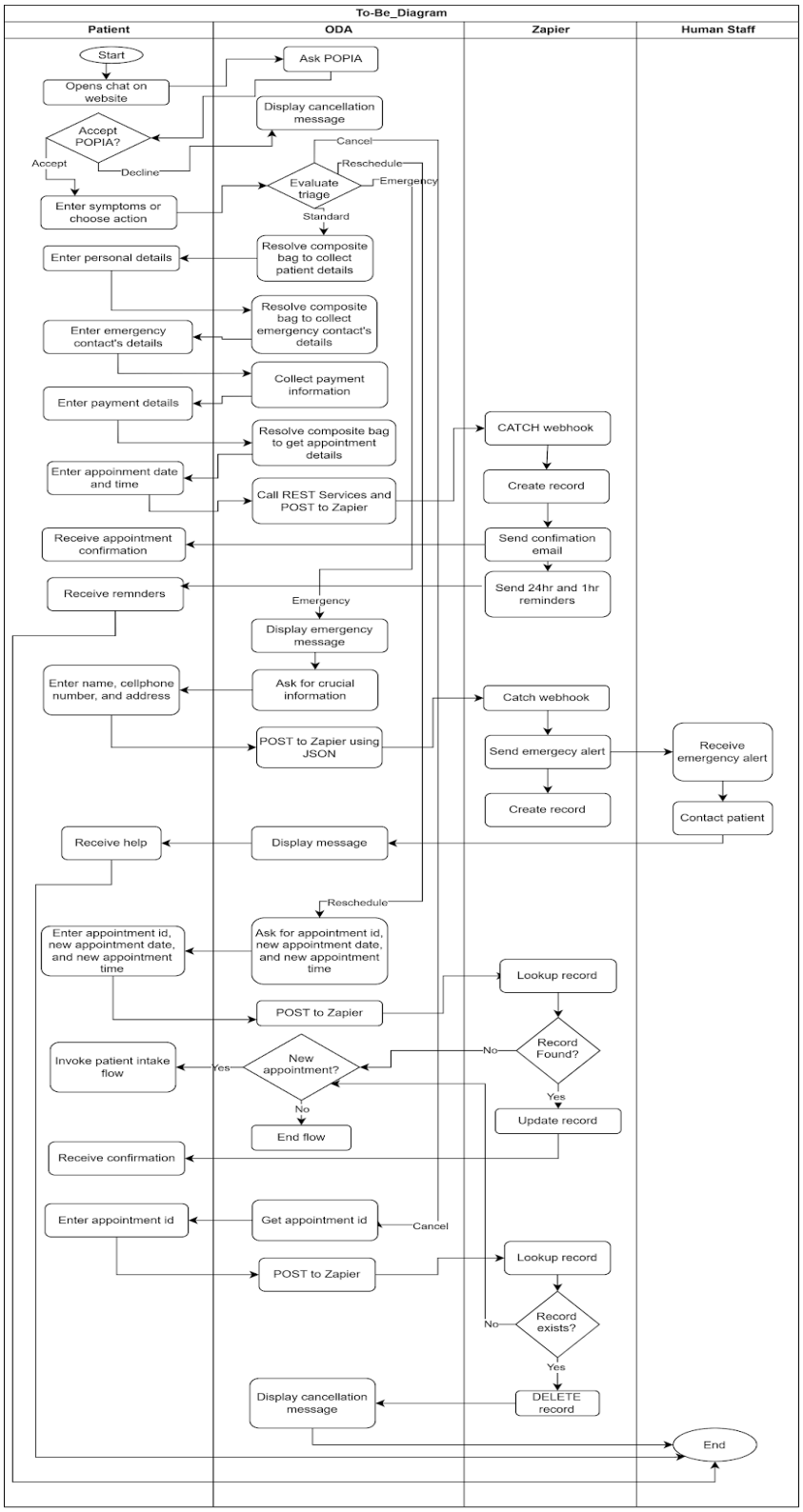
Req ID	Requirement Description	User Story ID	Implementation State / Component	Test Case ID	Status
	compliant data handling.		server-side credential configuration.		

SECTION 3: REVISED AS-IS AND TO-BE PROCESS MAPS (BPMN)

3.1 As-Is Clinical Process Map



3.2 To-Be Automated Clinical Process Map



3.3 Key Process Changes (Semester 1 vs. Semester 2 Revisions)

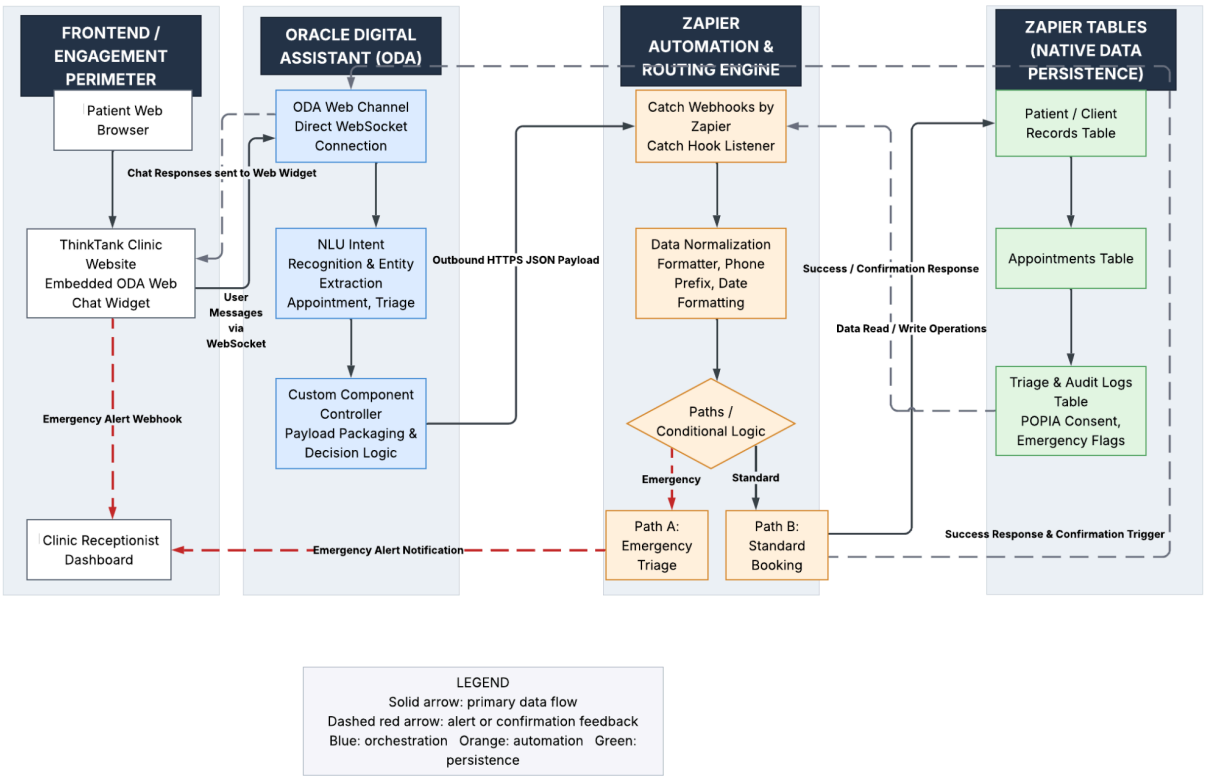
Design area	Semester 1: Initial design	Semester 2: Revised design	Reason for change
Patient information collection	The initial concept relied on webviews or separate input screens to collect patient information.	Revised to use native ODA Composite Bag entities and form-based slot filling, including cb_patient_essentials and cb_booking.	Keeps the patient intake process within the chatbot conversation and supports structured field collection and validation.
Conversation navigation	General chatbot conversation and basic service options were proposed.	ODA main menu routing and intent-based navigation were introduced, with System.Switch components used to direct patients to the appropriate flow.	Provides more structured navigation and separates intake, booking, enquiries and rescheduling paths.
Backend responses	Initial design relied more heavily on predefined or static chatbot responses.	Revised to use dynamic REST service calls, including the ODA System.CallRestService component, to communicate with backend services.	Allows the chatbot to process live requests and return information from connected systems rather than relying only on hard-coded responses.
Appointment information retrieval	Appointment information was expected to be managed through a	A lookup process was designed using an appointment record ID and Zapier Tables.	Supports looking up the corresponding appointment record using a unique identifier.

	general chatbot interaction.		
Patient intake workflow	Patient details were described as information to be collected by the chatbot.	A dedicated Patient_Intake_Flow was developed with consent handling, patient information collection and backend submission.	Separates the intake process into a defined workflow and improves traceability of collected information.
Emergency triage	Emergency handling was identified as a requirement in the proposed solution.	Red-flag detection and conditional triage routing were incorporated into ODA through the EvaluateTriage state and System.Switch logic.	Provides a dedicated route for emergency-related inputs rather than treating every patient request as a routine administrative interaction.
Emergency escalation	General staff involvement was proposed for urgent cases.	The revised architecture identifies a secure REST/Zapier notification workflow to forward relevant information to a nurse or authorised staff member.	Establishes a technical route for human intervention.
Messaging integration	Messaging channels were proposed to improve patient communication.	Zapier was used to send communications, including appointment reminders, via email.	Provides a dedicated communication integration.
Data storage and automation	Data processing and storage were initially described at a conceptual level.	Zapier workflows and Zapier Tables were incorporated into the backend design and	Provides a more defined mechanism for processing and

		used for appointment lookup.	recording appointment information.
Privacy and security	POPIA and patient data protection were identified as design considerations.	A consent step, backend authentication requirements and secure REST communication were incorporated into the revised design.	Makes privacy and security requirements part of the actual conversation and integration architecture.

SECTION 4: FINAL SYSTEM ARCHITECTURE & INTEGRATION DESIGN

4.1 Enterprise System Architecture Diagram



4.1.1 ODA to Zapier Webhook (GET Lookup)

Before new appointment records are created, ODA checks for possible existing records pertaining to the patient so as not to duplicate any data in the service database. ODA grabs conversation data, whereas Zapier does all its backend scripting for querying the database.

- Triggers: ODA makes a REST API call in a dedicated Zapier Lookup Catch webhook when a patient provides core identifying information in the cb_patient_essentials composite bag.
- Method: POST
- Headers: Content-Type: application/json
- Structure for Payload (JSON): ODA converts patient-identifying data into JSON format. FreeMarker syntax is used to catch null values and ensure proper string rendering.

Zapier's Operations for Processing:

1. When Zapier gets the data, it activates the "Finding Record" operation in the Patient Information table.
2. If the patient is located: Zapier sends the PID back to the ODA, so the bot can continue with the "Existing Patient" flow (e.g., rescheduling).
3. If the patient is not located: Zapier sends a "not found" message. ODA directs the user to proceed along the "New Patient" pathway.

4.1.2 ODA to Zapier Webhook (POST Intake Creation)

When the patient finishes going through the oral intake process (which includes consent, personal information, clinical issues, payment method, and making an appointment), then the data gathered will be sent to proceed with database entry creation.

- Trigger: ODA performs an API call to the Zapier "Standard Triage" Catch Webhook once the normal booking process is done.
- Method: **POST**
- Headers: Content-Type: application/json
- Payload Structure (JSON): ODA determines which variables and structure values need to be placed in a JSON payload. In order to make sure no other quote in the input text

causes a failure of our database due to misentry; the existing data object, which has the name 'json_string', will be used, and strict formatting of dates will be executed.

The Zapier Action:

1. In a situation where the information is provided to Zapier, it will process the payload and transmit it to the tables: Patient Basic Information and Appointments Table.
2. The data will also become consistent where relevant, which might require standardisation in terms of the date format, for example.
3. Lastly, the success code will be sent back to the ODA Web Channel in the shape of a final booking confirmation presented to a patient through the webchat widget.

4.2 Integration Specifications & Data Flow

A connection is created between the conversational AI, middleware, and storage layers through an event-driven, stateless architecture. The Oracle Digital Assistant (ODA) does not interact with the database but operates as the main orchestration engine that collects, validates, and converts user information into structured JSON payloads.

When a step in the conversation is reached, ODA makes a RESTful HTTP webhook call to Zapier. Zapier serves as the secure middleware that obtains the payloads and normalises the data (for instance, converting the date into the correct format) and executes the appropriate actions in the database and Zapier Tables.

The system can potentially utilise two different data flows depending on the outcomes of the clinical triage assessment conducted by the ODA NLU tool.

Data Flow A (Standard Booking):

If the individual showcases standard non-critical signs, the ODA will carry out the onboarding process (which includes gathering the required information about the user, emergency connection methods, and the time to receive the service). The data is sent through the Standard Triage Catch Webhook to Zapier. The latter then stores the received information in the patients

and appointment tables before sending a reply back to ODA to show a successful notification in the web chat widget.

Data Flow B (Emergency Triage):

As soon as dangerous symptoms, like "chest pain" or "shortness of breath", are picked up or a user selects the emergency route, ODA stops the usual booking flow. Payment and booking information are ignored in such cases. A crucial payload is instead sent to the Emergency Catch Webhook. The payload is collected and recorded by Zapier in the Triage and Audit Logs database and is also used to send a crucial webhook alert to the clinic's receptionist dashboard.

Data Integrity and POPIA Compliance:

Data flowing between ODA and Zapier is secured by means of HTTPS with TLS 1.3 encryption, as required by law. Furthermore, POPIA requires the principle of data minimisation to be adhered to, which means that the payloads permit only transmission of necessary information.

SECTION 5: CONVERSATION DESIGN

5.1 Intent & Entity Schema

The system is built with the assistance of the Oracle Digital Assistant (ODA), which serves to determine what the user means and collect the right entities. In order to handle extensive data acquisition effectively, ODA incorporates Composite Bag Entities (referring to many associated fields being put in one box) and Value List Entities (meaning that only given value could be used).

The below table displays the essential intents defined in the system as well as the entities documented throughout the communication.

Intent	Purpose	Entities/parameters captured
Patient_intake_flow	Starts the clinical intake, obtains the consent of the subjects in accordance with the regulations of POPIA and determines the route of the patient through the data collection and booking procedure.	popia_consent (String: Yes/No) cb_patient_essentials (Composite Bag) symptoms (Text) payment_type_choice (Value List) MedAidProvider (Value List) cb_booking (Composite Bag) cb_emergency_contact_info (Composite Bag)
Unresolved Intent	Fallback Intent: Refers to the responses given to unrelated questions, errors, or utterances that do not follow the protocol so that the chatbot does not break down.	It includes learning phrases for irrelevant sentences that are essential to start giving the fallback responses.

5.1.2 Composite Bag Information:

To ensure the smoothest possible transfer of data to Zapier, Composite Bags were created, including:

1. **cb_patient_essentials (identifying the patient)**
   1.1 patient_name (string)
   1.2 sa_id_no (string - 13 digits, verified by regex)
   1.3 DOB (date—format: YYYY-MM-DD)
   1.4 Patient email (string/email)
   1.5 Patient_number (string—SA phone number is normalised)

2. **cb_booking (scheduling)**
    2.1 Appointment_date (date)
    2.2 Appointment_time (time)

3. **cb_emergency_contact_info (emergency contact name)**
    3.1 Emergency_contact_name (string)
    3.2 Emergency_contact_number (string)
    3.3 Patient_address (string)

5.1.3 Value List Entities:

In order to prevent unapproved data from being entered in the data storage layer (Zapier), certain objects would be recognised as **value lists**, such as:

action_choice: EMERGENCY, STANDARD, RESCHEDULE, CANCEL

payment_type_choice: Cash, Medical Aid

MedAidProvider: Discovery, Bonitas, Momentum, Private Cash, etc.

symptoms: While the NLU heavily relies on text, it has also been trained to detect emergency words that initiate action for triage.

5.1.4 Input Validation & Entity Restrictions:

To ensure that the integrity of the database remains intact and that the database adheres to POPIA regulations, certain validation rules at the level of entity are employed:

- SA ID Number: The SA ID number should comply with the Regular expression of exactly a 13-digit pattern.
- Cell Numbers: Cell numbers should be in the correct South African format (which means the mobile number consists of 10 digits and starts with 0 and converted to the international format by Zapier, i.e., +27).
- Date of Birth: Dates should be in string-parsed format of YYYY-MM-DD to ensure no confusion while inserting data into the database.

5.2 Flow 1: POPIA Consent & Standard Intake (Happy Path)

In this section of the report, we provide information regarding the usual dialogue of a patient requesting a regular appointment. The patient finds that the past conversations with the Oracle Digital Assistant (ODA) have been useful because of its accurate combinations of POPIA consent and the gathering of important information by means of **Composite Bags**.

ODA State Progression:

Step	**ODA State**	**Activity and Transition Logic**
1	Start_Intake (Intent)	Initiated when the user says "I need a doctor."
2	Ask_POPIA_Consent (Ask a Question)	Shows the POPIA consent notice.
3	Check_POPIA (Switch)	If the answer is ‘No’, then go to the end of the session gracefully. If the answer is ‘Yes’, then go to the action menu.
4	Ask_Action (Ask a Question)	Prompting the user to choose action: EMERGENCY, STANDARD, RESCHEDULE, CANCEL.
5	Ask_Personal_Info (Resolve Composite Bag)	Request the user's name, SA ID, DOB, email and phone number.

6	Collect_Emergency_Contact (Resolve Composite Bag)	Request Name, Number and Address of emergency contact.
7	Payment_Choice (Ask a Question)	Choice of different paths: considering whether the user will use their medical aid or cash.
8	Collect_Booking (Resolve Composite Bag)	Request Appointment Date and Time.
9	Post_To_Zapier_Standard (Call REST Service)	Sends JSON payload to Zapier

Happy-Path Dialogue (Standard):

User: "I want to book an appointment."

Bot: "Welcome to the Clinical Patient Portal. Before we begin, please note that we collect personal information in accordance with the Protection of Personal Information Act (POPIA)."

Bot: "Do you consent to the processing of your personal data for the medical intake and booking process? (Reply 'Yes' or 'No')"

User: "Yes"

Bot: "Please enter your symptoms or select the action you want to do" (Displays Buttons: EMERGENCY, STANDARD, RESCHEDULE, CANCEL)

User: "STANDARD"

Bot: "Please enter your full legal name."

User: "Keamogetswe"

Bot: "Please enter your South African ID Number."

User: "0412216009099"

Bot: "Please enter your date of birth in this format: YYYY-MM-DD."

User: "2004-12-21"

Bot: "Please provide me with your email address."

User: "Keamogetswemokgoro@gmail.com"

Bot: "Please enter your South African Cell Number."

User: "0765286945"

Bot: "What is your emergency contact's full legal name?"

User: "Kea"

Bot: "Please enter your emergency contact's South African Cell Number."

User: "0799893777"

Bot: "What is your physical/residential address?"

User: "Kovacs"

Bot: "How will you be settling this consultation?" (Displays Buttons: Medical Aid, Cash)

User: "Cash"

Bot: "Which day would you like to request for your appointment?"

User: "10 December 2026"

Bot: "What time of day works best for you?"

User: "9:00 am"

Bot: "Thank you, Keamogetswe! Your intake form has been submitted successfully. Your appointment is confirmed for 10 December 2026 at 9:00 am."

System Integration:

Upon completion, the Custom Component Controller of ODA sends a POST request via REST API to the Standard Catch Webhook on Zapier and sets the triage_status to "STANDARD". Zapier records this data in Zapier Tables and sends a success response back to ODA, which finally shows the confirmation message in the web chat service.

5.3 Flow 2: Reschedule Appointment (Dynamic REST Lookup)

The actions of the bot are illustrated in the following sequence in the event that an old user wishes to change their previously reserved appointment.

Step	**ODA State**	**Activity and Transition Logic**
1	Start_Intake (Intent)	Triggered by the user picking the RESCHEDULE option.
2	Ask_Appointment_Number (Ask a question)	Requests a unique appointment ID from the user.
3	Perform_Lookup (Call REST service)	ODA issues a request via Zapier to locate the booking.
4	Evaluate_Lookup (Switch)	If the appointment exists, proceed to Ask_New_Time; if it does not, go to the system's error handler.

5	Ask_New_Time (Composite bag)	Prompts the user to give their new appointment date and time.
6	Post_To_Zapier_Reschedule (Call REST service)	Obtains a confirmation of the update from Zapier.

Path Dialogue (Reschedule):

User: "RESCHEDULE"

Bot: "Please enter the appointment number you got on your confirmation message for your appointment."

User: "01M6YBV3A07D2KVV5NA29THGMN"

Bot: "Please enter the date you'd like to reschedule your appointment to."

User: "10 November 2026"

Bot: "Please enter the time you'd like to schedule your appointment for."

User: "10:00 AM"

Bot: "Your record with appointment ID: 01M6YBV3A07D2KVV5NA29THGMN has been found and updated to Appointment_Date: 2026-11-10, Appointment_Time: 10:00:00"

System Integration:

ODA receives the new date/time and sends a payload to the Zapier "Reschedule" Webhook. Then, the Appointment ID is used by Zapier to find the relevant row in the Appointments Table and replace the old date/time information.

5.4 Flow 3: Emergency Triage Escalation

This flow illustrates the mechanism for clinical safety. When ODA's NLU engine detects potentially deadly symptoms or if the user selects the EMERGENCY button, it terminates the usual booking steps and moves on to an urgent situation.

Step	**ODA State**	**Activity and Transition Logic**
1	Start_Intake (Intent)	Triggered by user saying "Emergency"
2	Ask_POPIA_Consent (Switch Logic)	Standard POPIA consent agreement.
3	Ask_Action (Switch Logic)	The user selects EMERGENCY from the menu.
4	Emergency_Alert (Send Message)	The bot halts the standard booking process and showcases the critical alert message.
5	Collect_Essentials (Composite Bag)	Obtains name, SA ID, DOB, email, cell and address, which are then transmitted to the triage team.
6	Post_To_Zapier_Emergency (REST)	Delivers urgent payload to Zapier.
7	End_Flow (End)	The ODA session is closed, and the bot stops responding.

Path Dialogue:

User: "Emergency"

Bot: "Do you consent to the processing of your personal data? (Reply 'Yes' or 'No')"

User: "Yes"

Bot: "Please enter your symptoms or select the action you want to do" (Displays Buttons)

User: "EMERGENCY"

Bot: "CRITICAL ALERT: Based on your symptoms, immediate medical attention may be required. We are dispatching an emergency alert to our triage team now."

Bot: "Please enter your full legal name."

User: "Keamogetswe"

Bot: "Please enter your South African Cell Number."

User: "0765286945"

Bot: "Please provide me your address. It will be forwarded to the triage team."

User: "123 Main Street"

Bot: “Your details have been sent to our human assistant but if you are experiencing a life-threatening emergency, please call 10177 or visit the nearest hospital emergency room immediately.”

System Integration (Emergency Webhook Dispatch):

Upon the patient's selection of the EMERGENCY option and the subsequent provision of their details, Oracle Digital Assistant (ODA) makes the REST API POST request to the Zapier Emergency Catch Webhook.

The usual booking payload is not used by ODA; instead, it sends a higher priority JSON payload that includes details about the patient’s identity (patient_name, sa_id_no), details for contacting the patient (PatientNumber), and the patient address. In addition, it also marks the value of the triage_status variable as "CRITICAL / EMERGENCY" so that payment details and appointment information need not be requested.

Zapier Action:

Once this high-priority payload is received by Zapier, the normal logic for bookings is avoided, and it executes the following two things:

1. **Audit Logs:** It records this incident in the Triage & Audit Logs Table in Zapier Tables so there is permanent information on the emergency interaction.
2. **Staff Escalation:** An immediate WebSocket message is sent to the clinic receptionist in the form of a high-priority email containing details of the patient, such as their name and physical address, to enable timely action, such as telephonic intervention or emergency services dispatching if necessary.

SECTION 6: BUILD AND CONFIGURATION GUIDE

6.1 Oracle Digital Assistant (ODA) Setup

To create the agent, we set it up by adding states that run in a logical flow from start to end. Our Patient Intake and Appointment Management Bot has 2 distinct flows, namely **emergency** and **standard** triage. The following steps were taken to configure the Patient Intake agent on ODA;

1. **Skill Creation and Initialisation:**
    - Navigate to the ODA development environment and create a new skill.
    - Display Name: **Patient_intake**
    - Primary Language: English
    - Enable Visual Dialog as the dialog flow mode for easier node management and API integration mapping.

2. **Intent and NLP Configuration**

   To ensure the agent correctly identifies why the patient is interacting with it, primary intents were mapped.

   - **Created Intents:**
     - Initiate_Intake: Utterances include "I need to see a doctor", "Check in", "New patient", etc.
     - Emergency_Trigger: Utterances include "Chest pain", "Bleeding", "Can't breathe", and "Severe", etc.
   - Train the NLP model to ensure accurate confidence scoring between standard requests and potential emergencies.

3. **Composite Bag Entity (CBE) Configuration**

   To collect the patient's details dynamically without rigid, linear questioning, a **Composite Bag Entity** (often referred to casually as a "congregate bag") was configured. This allows the agent to extract multiple pieces of information from a single user statement or prompt for missing items one by one.

   - **Entity Name:** PatientDetailsBag
   - **Bag Items Configured:**
     - FullName (Type: System.Person): Prompts with "What are your first and last names?"
     - DOB (Type: System.Date): Prompts with "What is your date of birth?"
     - Symptoms (Type: System.Text): Prompts with "Please describe your symptoms."
     - TriageLevel (Type: String): A backend variable (not prompted to the user) to store the classification of "Emergency" or "Standard".

4. **Triage and Classification Logic**

   Configure business logic within the Dialogue Flow to classify the patient's status based on their input.

   - Add a 'Switch' or 'Condition' node following the extraction of the 'symptoms' item in the Composite Bag.
   - **Logic:** If the user's initial input matched the Emergency_Trigger intent, or if the Symptoms string contained high-risk keywords (e.g., "heart", "bleeding", "breathing"), the TriageLevel variable is updated to Emergency.
   - **Fallback:** If no critical keywords or intents are detected, TriageLevel defaults to Standard.
   - **Routing:** Emergency patients are routed to a node providing immediate critical instructions (e.g., "Please call emergency services or proceed to the nearest ER immediately"), while standard patients proceed to the standard Zapier handoff.

5. **API Connection to Zapier**

   Configure the REST API integration to pass the aggregated patient data out of ODA and into Zapier.

   - **Zapier Setup:** Generate a Catch Hook URL in Zapier to receive the incoming HTTP POST requests.
   - **ODA API Service Setup:**
     - Navigate to Settings > API Services in the ODA Skill.
     - Create a new REST service pointing to the Zapier webhook URL.
     - Method: POST
   - **Call REST Service Node (Dialogue Flow):**
     - Add a Call REST Service component at the end of the intake flow.
     - Map the PatientDetailsBag entity values into a JSON payload structure in the component's body property.

6.2 Zapier Webhook & Database Setup

1. Foundation: Database Setup (Zapier Tables)

Before configuring route workflows, build the target persistence schema using **Zapier Tables** (build from scratch):

- **Schema Definition:** Define explicit fields and data types for each table entity (Patient Details, Appointments, Emergency Triage Logs).
- **Primary Key Generation:** Assign a unique record identifier using the built-in record_id formula to guarantee entity uniqueness for every new entry.

2. Workflow Routes Setup (Zapier Webhooks)

Route A: Standard Intake & Booking Route

1. **Trigger:** Initialise a **Catch Webhook** trigger to receive the inbound ODA JSON payload containing intake and slot-filling variables.
2. **Data Formatting:** Insert a **Formatter** step to convert date/time strings to local time zones (UTC→SAST) and calculate relative offsets.
3. **Database Insertion:** Add a **Create Record** step targeting the Appointments table, mapping dynamic JSON fields (patient_name, sa_id, appt_datetime) to table columns.
4. **Calendar Event:** Add a **Google Calendar** step to generate the clinical appointment, mapping start/end times and attending practitioner emails.
5. **Confirmation Email:** Add an **Email (SMTP)** step to dispatch an immediate booking confirmation to the patient with mapped dynamic details.
6. **Scheduled Reminders:** Add **Delay Until** logic is set to execution thresholds (e.g., T−24h and T−1h), followed by automated reminder email triggers.

Route B: Emergency Triage Route

1. **Trigger:** Initialise a **Catch Webhook** trigger to capture high-priority emergency payloads from ODA's triage flow.
2. **Audit Logging:** Execute a **Create Record** step in the Emergency Triage Logs table to store patient contact details, location, and symptoms.

3. **Priority Alert Dispatch:** Immediately trigger an urgent notification email/SMS directly to the human clinical staff console (Emergency Triage Alert).

Route C: Appointment Reschedule Route

1. **Trigger:** Initialise a **Catch Webhook** trigger receiving the target appointment_id and updated date/time request from ODA.
2. **Record Lookup:** Execute a **Find Record** step in Zapier Tables using the provided appointment_id.
3. **Conditional Logic:**
    - **If Found:** Execute an **Update Record** step with new date/time values, update the calendar event, and dispatch a reschedule confirmation email.
    - **If Not Found:** Return a found: false response to ODA to trigger the *"Invalid ID / Retry"* fallback path.

Route D: Appointment Cancellation Route

1. **Trigger:** Initialise a **Catch Webhook** trigger receiving the appointment_id deletion request from ODA.
2. **Record Lookup:** Execute a **Find Record** step in Zapier Tables to verify record existence.
3. **Conditional Logic:**
    - **If Found:** Execute a **Delete Record** step in Zapier Tables, remove the corresponding Google Calendar event, and send a cancellation receipt email.
    - **If Not Found:** Return an error status to ODA to prompt user re-entry.

6.3 Website Creation and Publishing

To give patients a public place to use the agent, we built a simple website that hosts the Patient Intake chat widget. The site is hosted on GitHub Pages and connects to the ODA skill through a Web channel. The following steps were taken to create and publish it.

6.3.1. Foundation: ODA Web Channel Setup

Before building the page, we created the channel that lets the website talk to the skill.

- Navigate to Development > Channels in ODA and create a new channel of type Web.
- Route the channel to the Patient_Intake skill.
- Record the channel ID and the ODA instance address, as the website needs both to connect.
- Leave the Allowed Domains field empty until the website address is known

6.3.2. Website Structure and Files

The website is made of a small set of static files, organised as follows:

- **`Index.html`** : The main page, containing the layout, the introduction text, and the placeholder where the chat widget loads.
- **`Styles.css`** : The colours, fonts, spacing, and responsive layout, so the page works on both desktop and mobile.
- images/: The logo and any images shown on the page.
- **`scripts/chat-config.js`:** The chat settings and the code that starts the widget.

6.3.3. Chat Widget Integration (Oracle Web SDK)

- Add the Oracle Web SDK link to **`index.html`** with a script tag, placed before the line that loads **`chat-config.js`.**
- In **`chat-config.js`,** set the instance address and the channel ID recorded in Step 1, and start the chat widget with these values.
- The Secret Key is never placed in any website file. The repository is public, so anything stored in it can be seen by anyone.

6.3.4. Publishing with GitHub Pages

- Create a **GitHub** repository and upload the website files.
- Under 'Builder and Development', choose to deploy from a branch, select the **main** branch and **/ (root)** folder.
- **GitHub** displays a live address
- Return to ODA channels and paste the live address

5. Testing the Live Address

The site was tested on the live address, not only on VS Code (GO Live):

- **Design:** The page loads correctly with the right layout, colours, and fonts on desktop and mobile.
- **Images:** The logo and all other images displayed.
- **Chat:** The widget opens, the agent greets the user, and it replies. Both flows (standard and emergency triage) were run through to the end.

SECTION 7: CODE DOCUMENTATION AND REPOSITORY

7.1 Repository Details

- **GitHub Repository URL:** https://github.com/QhayiyaMani/IFS325_Group14_UseCase4.git
- **Repository Structure:**
  - /oda-bot-export/ — ODA Bot Definition & Flow YAML
  - /middleware/ — REST API handler scripts
  - /zapier-templates/ — JSON payloads & webhook schema definitions
  - /docs/ — BPMN diagrams & architectural schemas

7.2 Zero-Credential Compliance Verification

Security Audit Confirmation: All API keys, authorisation tokens, Infobip credentials, and database connection strings have been removed from source files and stored strictly in environment variables (.env).

SECTION 8: TESTING AND QUALITY RECORD

8.1 Test Plan Summary

The testing phase evaluated the functionality, reliability, performance, and end-to-end integration of the **Patient Intake and Appointment Management Bot**.

- **Test Suite Scope:** Executed **18 comprehensive test cases** covering core operational workflows: patient registration, slot availability, PII/medical aid collection, booking confirmations, dynamic reminders, rescheduling, cancellations, and data persistence across integrated systems.
- **Resilience & Edge-Case Validation:** Conducted robust stress and error-handling scenarios using malformed inputs, incomplete data, duplicate requests, and simulated API/integration failures.
- **Execution & Verification:** Validated end-to-end conversational flows against backend data states in Zapier Tables and middleware webhooks, recording outcomes directly into the **Test Execution Records (Section 8.2)** to ensure full functional compliance.

8.2 Test Execution Record

Test ID	Category	Scenario / Input	Expected Result	Actual Result	Pass/Fail
E2E-01	Website and Chatbot Access	The patient accesses the clinic's Google Sites website and opens the Oracle Digital Assistant chatbot.	The chatbot successfully loads and allows the patient to start a conversation.	The chatbot loaded successfully.	Pass
E2E-02	New Patient Registration	The new patient provides their name, surname, date of birth and contact details through the chatbot.	Patient information is captured correctly and stored in the appropriate Oracle patient record.	The patient registered successfully.	Pass
E2E-03	Existing Patient Identification	The existing patient provides a valid Patient ID and requests an appointment.	The system identifies the existing patient and retrieves the correct patient information from Oracle.	Patient identified successfully.	Pass
E2E-04	Appointment Type and Provider Selection	Patient selects an appointment type and requests an appointment with a specific healthcare provider.	Correct appointment type and provider are selected and associated with the booking.	Appointment type recorded correctly.	Pass
E2E-05	Appointment Availability	Patient requests a specific appointment date and time.	System checks Oracle appointment availability and returns the correct available or unavailable response.		Fail

E2E-06	Appointment Booking	The patient provides all mandatory booking information and confirms the appointment.	The appointment is successfully created in Oracle and linked to the correct patient record.	Appointment booked successfully.	Pass
E2E-07	Appointment Confirmation	The patient completes a valid appointment booking through the chatbot.	Patient receives a confirmation containing the appointment date, time and provider details.	Confirmation sent successfully.	Pass
E2E-08	Appointment Rescheduling	The patient provides a valid appointment ID and requests a new available date and time.	The existing appointment is updated with the new date and time in Oracle.	Appointment rescheduled successfully.	Pass
E2E-09	Appointment Cancellation	The patient provides a valid appointment ID and requests cancellation.	The appointment status changes to Cancelled in Oracle, and the appointment slot is released.	Appointment cancelled successfully.	Pass
E2E-10	Oracle Integration and Data Accuracy	Complete a booking through the chatbot and inspect the corresponding patient and appointment records in Oracle.	Correct patient, appointment, provider and appointment status information is stored in Oracle without duplicate or incomplete records.		Fail

TC-01	Special Character Validation	The patient enters a name containing a hyphen, apostrophe or accented character.	The chatbot accepts valid characters and captures the patient's name correctly.	Special characters are accepted.	Pass
TC-02	Date Format Validation	Patient provides a date in an unexpected format, such as 12/15/2026.	The chatbot correctly asks the patient to confirm it before processing.	Date format recognised.	Pass
TC-03	Multiple Information Extraction	The patient provides their name, patient ID and preferred appointment date in one message.	The chatbot asks the patient to clarify missing details in the following messages.	Information not recognised	Pass
TC-04	Empty Message Handling	The patient sends a blank message or a message containing only spaces.	The chatbot does not process the message as a valid request and prompts the patient to provide information.	Empty message rejected.	Pass
TC-05	Invalid Appointment Type	The patient enters an appointment type that is not included in the configured appointment options.	The chatbot does not allow the unsupported selection, and the patient must choose a valid appointment type.	Invalid appointment type rejected.	Pass
TC-06	Missing Required Information	The patient attempts to complete a booking without providing all mandatory information.	The chatbot identifies the missing details and does not move on until the required information	Missing information identified.	Pass

			is given before proceeding.		
TC-07	Duplicate Booking Prevention	The patient submits the same appointment booking request twice.	The chatbot identifies the potential duplicate and prevents an unintended second appointment.		Fail
TC-08	Error Handling and Escalation	The patient submits a complex request that the chatbot cannot handle, or an Oracle integration error occurs.	The chatbot handles the situation appropriately, does not provide false booking confirmation, and directs the patient to the appropriate support channel.		Fail

---

SECTION 9: USER GUIDE NOTES

This section explains how patients use the prototype on the website web-chat, what the assistant expects them to type, and what to do when something goes wrong. It is written for patients and for the staff who support them. The assistant handles administrative tasks only and cannot give medical advice. WhatsApp is not part of the delivered product and is covered as a future development in section 11.2.

9.1 How End Users Interact Via The Web Chat (Standard Intake Conversation)

The **Patient Intake and Appointment Management Bot** is accessed directly through the clinic's web portal via an embedded, responsive chat widget powered by the Oracle Digital Assistant (ODA) Web SDK. Users do not need to download third-party messaging apps or register external accounts.

1. System Requirements & Access

- **Supported Devices:** Desktop computers, laptops, tablets, and smartphones.
- **Hosting Environment:** The clinic web portal and conversational widget are hosted via GitHub Pages directly from a secondary project repository.
- **Browser Compatibility:** Latest versions of Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge with JavaScript enabled.
- **Access URL:** https://4349546-boop.github.io/thin-tank-site/oda-website/. Navigate to the clinic web portal and click the floating chat icon in the bottom-right corner of the page to initialise the session.
- **Initialisation:** The floating chat widget must be selected once the site is opened and triggered using utterances such as "I need a doctor," "I want to book an appointment," or by stating whichever symptom a patient is experiencing.

2. General Navigation & Interaction Tips

- **Starting a Session:** Click the chat bubble icon. The bot automatically presents a welcome prompt and the mandatory POPIA Privacy Notice.
- **POPIA Compliance Gate:** You must accept the POPIA data consent terms (Accept) to proceed with automated intake. Selecting 'Decline' exits the automated chat and provides alternative non-bot contact options (e.g., the reception phone number).
- **Conversational Slot-Filling:** The bot uses multi-turn conversations to collect personal, emergency, and booking details. Provide clear, standard formatting when prompted (e.g., 13-digit South African ID numbers and 10-digit phone numbers).
- **Main Menu Options:** At any point during a standard interaction, you can select from the core menu pathways:
    - **Book Appointment:** Initiate new patient registration and schedule a clinic slot.

    - **Reschedule Appointment:** Update an existing booking using your assigned Appointment ID.
    - **Cancel Appointment:** Remove an existing booking from the clinic schedule using your Appointment ID.
- **Emergency Override:** If you are experiencing acute, severe symptoms, state your condition immediately (e.g., "severe *chest pain"* or *"heavy bleeding"*). The bot automatically suspends standard booking and routes you to the Emergency Triage protocol with immediate instructions and urgent clinical staff escalation.
- **Correcting Errors:** If an invalid appointment ID or improperly formatted phone number is entered, the bot alerts you (ID Not Found / Invalid Input) and prompts you to re-enter the detail without restarting the entire conversation.

9.2 Expected Input And Validation

Field/Feature	**Allowed Input Format**	**Data Type/Pattern**	**Validation Rule & Error-Handling**
POPIA Consent	Type ‘Yes’ or 'No' (in that exact format with a leading capital letter, i.e., no lowercases are accepted).	String	Constraint: Binary selection required. Validation: If invalid text is typed, ODA re-displays the consent buttons.
Full Name	Freet text (e.g., John Doe).	String ([A-Za-z\s]+)	Constraint: Minimum 2 characters; no numbers or special symbols. Error: "Please enter a valid full name using letters only."
South African ID	13-digit numeric string (e.g., 9501015000080)	String (^\d{13}$)	Constraint: Must be exactly 13 digits and satisfy Luhn checksum validation. Error: "Invalid SA ID number. Please enter a valid 13-digit identity number."

Field/Feature	Allowed Input Format	Data Type/Pattern	Validation Rule & Error-Handling
Cellphone Number	10-digit SA mobile format (e.g., 0712345678 or +27712345678).	String	Constraint: Must start with a valid SA mobile prefix (06, 07, or 08) and total 10 digits. Error: "Invalid mobile number. Format must be 10 digits starting with 06, 07, or 08."
Email Address	Standard email (e.g., patient@example.com)	String	Constraint: Standard syntax containing @ and valid domain extension. Error: "Invalid email format. Please enter a valid address (e.g., name@domain.com)."
Appointment ID	Only accepts the one located in the confirmation email.	String	Constraint: Must exist in the Zapier Tables database (Record Found? = Yes). Error: "ID Not Found / Invalid Input. Please check your reference and re-enter."
Appointment Date/Time	Formatted string (YYYY-MM-DD).	DateTime / ISO 8601	Constraint: Must be during operating hours and at least 24 hours in the future. Error: "Selected slot unavailable or outside operating hours. Please pick another time."

9.3 Exception & Error Handling (Troubleshooting)

Section 9.3 defines the system's error handling strategies, exception classification, and step-by-step troubleshooting procedures across the Oracle Digital Assistant (ODA) front-end, Zapier middleware integration layer, and database persistence operations.

1. System Exception Classification & Fallback Protocols

The bot architecture utilises a defence-in-depth approach to manage exceptions, ensuring that system failures never crash the conversational interface or lose patient intake data.

A. Input Validation Exceptions (Front-End)

- **Trigger:** Patient enters invalid SA ID numbers, malformed phone numbers, non-existent dates, or unformatted text strings during Composite Bag slot filling (cb_patient_essentials).
- **Handling Protocol:** ODA intercepts the invalid input at the conversational entity level before executing any outbound API call. The bot displays a clear error explanation (e.g., *"Invalid SA ID number. Please enter a valid 13-digit identity number."*) and re-prompts the user for that specific slot without resetting the rest of the form.

B. Database Lookup & Missing Record Exceptions

- **Trigger:** The patient attempts to reschedule or cancel an appointment using an invalid, expired, or mistyped appointment ID.
- **Handling Protocol:** Zapier executes a Find Record query against **Zapier Tables**. If no matching record is found (Record Found? = No), Zapier sends a found: false response back to ODA. ODA displays an explicit notification (*"ID Not Found / Invalid Input"*) and prompts the user to re-enter their ID or contact human clinic staff.

C. Middleware & Network Failure Exceptions

- **Trigger:** Webhook HTTP timeouts, transient network drops, or Zapier service interruptions during POST operations (System.CallRestService).
- **Handling Protocol:** ODA utilises built-in HTTP retry policies (3 automatic retry attempts with exponential backoff). If the connection permanently fails, ODA triggers its system

error handler: *"We are currently experiencing technical difficulties saving your appointment. Please contact the clinic directly at 021-XXX-XXXX."*

D. Emergency Override & Safety Exceptions

- **Trigger:** System fails to recognise an emergency triage condition due to unexpected medical terminology, or patient drops connection mid-triage.
- **Handling Protocol:** The bot maintains an explicit, static fallback button (*"Emergency / Urgent Care"*) accessible at all stages of the chat menu. If an emergency intent is triggered, Zapier writes an audit record to Emergency Triage Logs and immediately dispatches a priority email/SMS alert to human staff, independent of the patient session state.

2. Troubleshooting Matrix for Operators & Support Staff

The following matrix provides clinical administrative staff and system administrators with immediate resolution steps for common technical and operational issues.

Symptom/Issue	**Primary Cause**	**Diagnostic Check**	**Resolution/Corrective Action**
The chat widget does not load on GitHub Pages.	GitHub Pages deployment build delay, incorrect script path in index.html, or invalid ODA Channel ID.	Firstly, check the GitHub Actions / Pages build status under repository settings. Secondly, open the browser developer console (F12) to check for 404 script loading errors or blocked mixed content.	Verify the GitHub Pages source is set to main branch, ensure the ODA Web SDK script links in index.html are correct, and confirm the Channel ID is active.
User stuck in infinite POPIA loop.	Misconfigured intent routing or loop line on POPIA 'Decline' selection.	Test the 'Decline' path in the ODA Dialog Tester console.	Ensure the Decline gateway arrow points directly to Display Exit Message and routes into an End Event (terminating the chat gracefully).

Symptom/Issue	Primary Cause	Diagnostic Check	Resolution/Corrective Action
"ID Not Found" error during Reschedule / Cancel workflows.	Typo in multi-character Appointment ID or record missing from database.	Perform a manual lookup in Zapier Tables (Appointments) using the patient's ID.	Ask the patient to verify the ID from their booking email. If valid but missing, re-enter details manually or prompt retry via bot.
Confirmation or reminder email not received.	Zapier Catch Webhook failure, email formatting syntax error, or invalid patient address.	Inspect Zapier Task History for execution failures on the target Zap.	Firstly, confirm patient email syntax in Zapier Tables. Secondly, verify SMTP/email host configuration steps in Zapier.
The emergency alert email/SMS is not arriving at the clinic.	Spam filter blocking or misconfigured staff emergency email address in Zapier.	Check Zapier Task Log for the Emergency Triage Zap outbound execution steps.	Whitelist clinic emergency emails (e.g., triage@clinic.uwc.ac.za) and verify recipient fields in Zapier Action steps.
Webhook 401 / 403 Unauthorised Error	Revoked Zapier Catch Webhook endpoint or incorrect REST URI.	Inspect the ODA System.CallRestService settings and target REST parameters.	Verify that the target Zapier Webhook URL matches environment variable definitions without hardcoding secret keys.
Inputs repeatedly rejected (SA ID / Mobile Number)	User input failed Composite Bag regular expression syntax checks.	Observe chat output during slot filling (cb_patient_essentials).	Input must strictly adhere to valid formats (13-digit SA ID satisfying Luhn check; 10-digit phone starting with 06, 07, or 08).

SECTION 10: CLOSED-OUT RISK REGISTER

Risk ID	Identified Risk	Impact / Severity	Mitigation Strategy	Actual Outcome & Status
R-01	Chatbot integration with Google Sites may fail or display incorrectly.	High. Patients may be unable to access the chatbot through the website.	Verify chatbot embedding, website accessibility, and compatibility.	Website and chatbot integration checked during testing.
R-02	Oracle Digital Assistant may misunderstand patient requests.	Medium. Patients may receive irrelevant responses or experience interrupted conversations.	Configure suitable intents, conversation flows, and fallback responses.	Conversation handling scenarios included in testing.
R-03	Appointment information may not be stored correctly in the database.	High. Patient and appointment records may become inaccurate or incomplete.	Validate database fields and verify stored information.	Data storage scenarios included in testing.
R-04	Appointment scheduling conflicts may occur.	High. Multiple patients could be assigned the same appointment slot.	Implement appointment availability checks before confirming bookings.	Scheduling conflict scenarios included in testing.
R-05	Patients may provide incorrect personal information.	High. Incorrect information could affect patient identification and records.	Apply input validation and mandatory field checks.	Patient information validation is included in testing.

Risk ID	Identified Risk	Impact / Severity	Mitigation Strategy	Actual Outcome & Status
R-06	Automated appointment reminders may not be delivered at the correct time.	Medium. Patients may forget appointments, increasing no-show rates.	Configure reminder schedules and verify notification triggers.	Reminder scenarios included in testing.
R-07	Patients may not understand how to navigate the chatbot.	Medium. Patients may abandon the booking process.	Use clear instructions, simple language and structured conversation options.	Chatbot interaction and usability scenarios included in testing.
R-08	Project delays caused by configuration and integration dependencies.	Medium. Could affect development progress and delivery timelines.	Prioritise core chatbot functionality and allow time for integration and testing.	Project deliverables and development activities managed against the project requirements.

SECTION 11: KNOWN LIMITATIONS & PRODUCTISATION ROADMAP

11.1 Known System Limitations

This section lists what the delivered prototype does not yet achieve, as the initial scope states. The prototype is the web-based chatbot, published on Google Sites websites: a patient chats with the patient_intake skill on Oracle Digital Assistant (ODA), which runs a standard booking flow, an emergency booking flow, and a series of questions from the new patients, and ODA passes the intake data to Zapier through the REST webhook. Infobip was integrated and tested during development, but WhatsApp is not part of the delivered scope and is suggested for future

development.Each limitation is rated (High, Medium, Low) by its effect on the development of the prototype.

A.

LIMITATIONS	IMPACT	SEVERITY
The web chat is the only channel. Patients without a device or data cannot use it.	Patients who have no digital gadgets, no data, and an unreliable connection cannot use the service.	Low
Patients identify themselves by typing a name and date of birth, with no verification.	Someone could book or change an appointment for another person.	Medium
Needs an internet connection	The chat cannot load offline or on a weak connection, and the assistant is unavailable if Oracle services are down.	Medium
The chat only understands what the skill was trained for.	Unusual requests get the standard error message in return.	High
Sessions and tokens expire.	A patient who leaves the conversation midway may have to start all over again.	Medium

B. WhatsApp Channel Through Infobip (Future Development)

These limits were discovered while integrating Infobip with ODA during development. The delivered web chatbot does not depend on them, but each must be solved before WhatsApp is added.

LIMITATION	IMPACT	SEVERITY
The Infobip free trial only delivers messages to the phone number verified at sign-up, with a limited number of free messages and an end date.	Only the account owner can interact with the bot, so there is no pilot with real patients and limited testing.	High
The middleware handles text, buttons, lists, and location. Images, documents, and voice notes from patients are not processed, and ODA cards are sent as a sequence of separate messages.	Patients cannot send photos or voice notes, and card layouts are simplified.	Medium
Infobip is not a native ODA channel, so a middleware service must sit between them.	The middleware has to be built, hosted, secured, and monitored. It adds second place, where the product can fail, outgoing hosting costs, and dependence on whoever maintains it after.	High

ODAs session timeout and WhatsApp 24-hour window run separately.	A conversation can be live in one system and expired in the other.	Medium

11.2 Future Productisation Roadmap (Think Tank Handover)

To transition the **Patient Intake and Appointment Management Bot** from its current capstone deployment into an enterprise-grade, scalable SaaS solution for clinical operations, a structured productisation roadmap has been established. This strategy focuses on enhancing architectural resilience, expanding multi-channel reach, deepening EHR integration, and integrating predictive AI capabilities.

1. Technical & Architectural Scalability

- **Standardised Healthcare Interoperability (HL7 / FHIR Standards):**

  Future iterations will transition from Zapier Tables middleware to native **Fast Healthcare Interoperability Resources (FHIR)** APIs and **HL7 messaging protocols**. This will enable direct, real-time synchronisation with major Electronic Health Record (EHR) platforms (e.g., Epic, Cerner, or local hospital management systems), eliminating intermediate database synchronisation steps.

- **Serverless Microservices Layer:**

  To handle higher concurrent patient volumes during peak clinic hours, the integration middleware will evolve from Zapier webhooks to containerised serverless microservices hosted on AWS Lambda or Azure Functions. This guarantees auto-scaling capability, sub-second API latency, and lower operational overhead.

2. Expanded Patient Accessibility & Engagement

- **Multi-Channel Expansion (WhatsApp Business API & USSD):**

  While the current web-based ODA Web SDK widget provides zero-install browser access, future productisation will extend the conversational engine to **WhatsApp Business API** and **USSD gateways**. This expands access to patients in low-bandwidth regions or those using feature phones, ensuring equitable healthcare access across diverse demographic groups.

- **Multilingual Natural Language Processing (NLP):**

  To serve South Africa's diverse population, ODA's intent engine will be trained on multilingual datasets, enabling seamless conversational switching between English, isiXhosa, and Afrikaans.

- **Voice-Enabled Conversational AI:**

  Integrating speech-to-text (STT) and text-to-speech (TTS) engines directly into the Web SDK widget will support hands-free intake for visually impaired or motor-impaired patients, fulfilling universal accessibility standards (WCAG 2.1 AA).

3. Advanced Clinical AI & Predictive Analytics

- **Predictive No-Show & Booking Optimisation:**

  Integrating machine learning models trained on historical attendance patterns will allow the system to predict high-risk "no-show" appointments. The bot can then automatically dispatch targeted early confirmation prompts or offer released slots to standby patients.

- **Symptom Intelligence & Dynamic Triage Assistance:**

  Expanding the emergency triage module with verified clinical symptom-checking models will allow the bot to provide more granular, pre-consultation summaries to human clinical staff, streamlining patient intake upon physical arrival at the clinic.

4. Enterprise Governance, Security & Compliance

- **Advanced POPIA / HIPAA Compliance Auditing:**

  Future production releases will incorporate automated field-level encryption (FLE) for all stored patient data at rest, immutable audit logging via dedicated security vaults, and automated data retention policies that purge or anonymise PII in accordance with national health record retention acts.

SECTION 12: PROJECT GANTT CHART (PLANNED VS. ACTUAL)

12.1 Project Lifecycle Overview & Execution Summary

The development of the **Patient Intake and Appointment Management Bot** for the UWC Health Clinic followed an Agile-Iterative lifecycle spanning five distinct project phases: Requirements Engineering, Architectural Design, Sprint Iteration & Prototyping, Integration & System Testing, and Deployment Documentation.

While the initial project baseline established aggressive timelines for simultaneous multi-channel deployment (including native mobile application wrappers and multi-platform messaging integrations), empirical testing during Sprint 2 necessitated strategic scope refinements. By adhering to Agile principles, the team prioritised high-value clinical intake, zero-credential security compliance, and robust data persistence over redundant channel exposure.

12.2 Scheduled Variances & Timeline Analysis

A comparative analysis between the baseline Gantt chart and actual task execution reveals three primary timeline variances:

Sprint 1: Extended NLU & Composite Bag Modelling (+4 Days Variance)

- *Baseline Schedule:* 5 Days allocated for initial Oracle Digital Assistant (ODA) intent and entity training.
- *Actual Execution:* 9 Days consumed.
- *Variance Driver:* Designing dynamic multi-turn slot filling via Composite Bags (cb_patient_essentials and cb_emergency_contact) required extensive regular expression tuning for South African ID numbers (Luhn algorithm validation) and mobile number formats (^0[6-8]\d{8}$). This upfront effort significantly reduced front-end exception rates during final user testing.

Sprint 2: Middleware Architecture Transition (+3 Days Variance)

Baseline Schedule: 4 days allocated for custom Node.js middleware hosting on a third-party cloud VM.

- *Actual Execution:* 7 days consumed.
- *Variance Driver:* Transitioning from custom-coded middleware to **Zapier Catch Webhooks** and **Zapier Tables** introduced an initial learning curve but eliminated long-term server maintenance overhead and security vulnerabilities associated with self-managed cloud endpoints.

Sprint 3: Compliance & Security Hardening (-2 Days Variance)

- *Baseline Schedule:* 6 Days allocated for security reviews and POPIA alignment.
- *Actual Execution:* 4 Days consumed.
- *Variance Driver:* Implementing centralised server-side environment variables and standardising quick-reply consent gates early in ODA accelerated the final security audit, allowing the team to recover schedule buffer prior to project submission.

12.3 Strategic Technical Pivots

During system implementation, Group 14 evaluated technical trade-offs and executed three key architectural pivots to optimise performance, maintain strict zero-credential compliance, and ensure seamless delivery:

Pivot 1: Web-Based ODA Chatbot via GitHub Pages vs. Native Mobile/Messaging Apps

- **Original Plan:** Deploy the bot simultaneously across third-party messaging channels (e.g., WhatsApp Business API) and embed it inside a standalone hybrid mobile application.
- **Pivot Rationale:** WhatsApp API integration introduced significant latency during dynamic slot filling, complex licensing dependencies, and external credential management challenges. Conversely, deploying the conversational interface via the **ODA Web SDK embedded on a GitHub Pages web portal** ensured zero-install access for patients across both desktop and mobile browsers, eliminated third-party API costs, and provided full control over CSS/UI styling and privacy disclaimers.

Pivot 2: Zapier Tables Relational Storage vs. External SQL Server

- **Original Plan:** Direct database connection from ODA to an external hosted MySQL database instance.
- **Pivot Rationale:** Connecting ODA directly to a relational database required maintaining static database credentials, database drivers, and complex firewall rules. Pivoting to **Zapier Catch Webhooks paired with Zapier Tables** established a stateless, decoupled architecture where Zapier handles authentication, payload mapping, CRUD persistence, Google Calendar scheduling, and automated email dispatches via serverless webhooks.

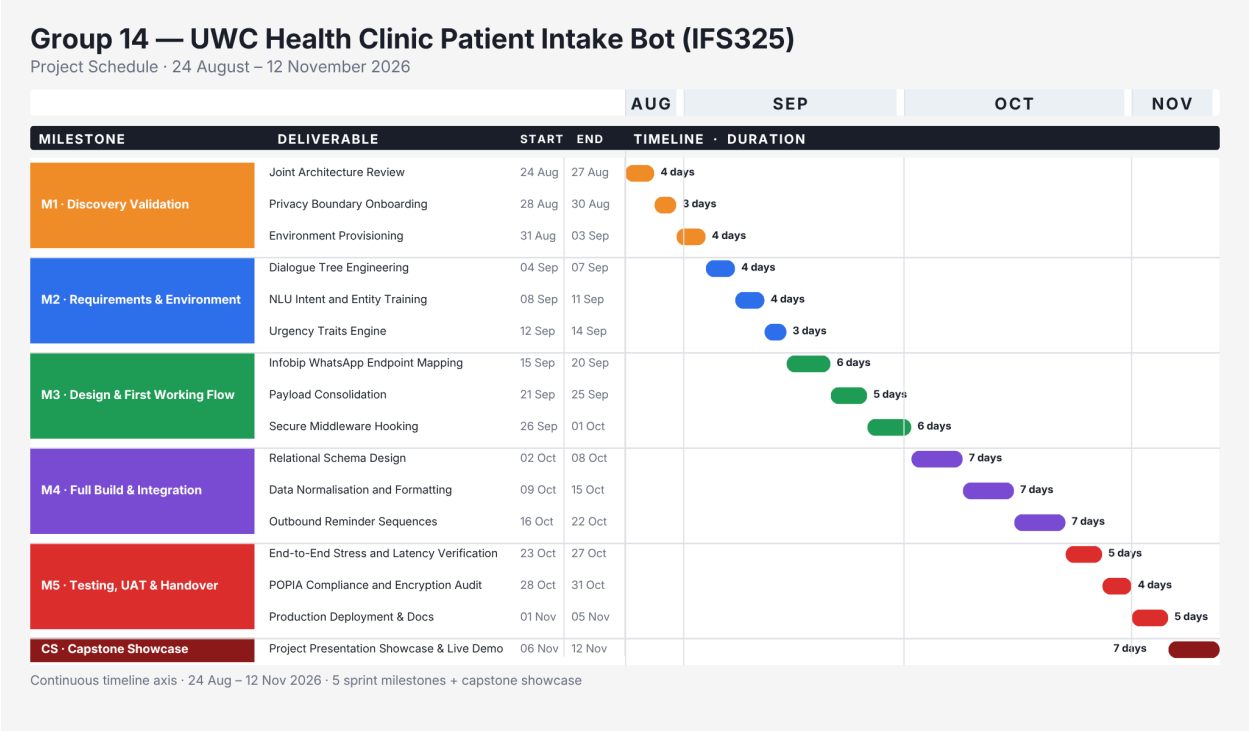
Pivot 3: Strict Environment Variable Injection vs. Hardcoded Webhook Endpoints

- **Original Plan:** Hardcoding destination REST endpoint URLs directly inside ODA dialog state transitions.
- **Pivot Rationale:** To enforce strict adherence to the university's zero-credential policy and POPIA data protection guidelines, all webhook endpoints and communication URIs

were refactored to consume server-side environment variables (${ZAPIER_WEBHOOK_URL}). This architectural constraint guarantees that no production secrets or live database links are exposed in source control repositories or project documentation exports.

- .12.4 Lessons Learned & Operational Recommendations
- **Agile Scope Management:** Prioritising core CRUD operations (booking, lookup, reschedule, cancel) and emergency triage over auxiliary channel integrations yielded a more stable, production-ready core system.
- **Early Security Guardrails:** Enforcing zero-credential standards from Sprint 1 prevented costly refactoring during final repository preparation.
- **Decoupled System Architecture:** Utilising Zapier as an intermediary integration layer insulated the conversational front-end from backend data format changes, allowing independent testing and debugging of business logic.

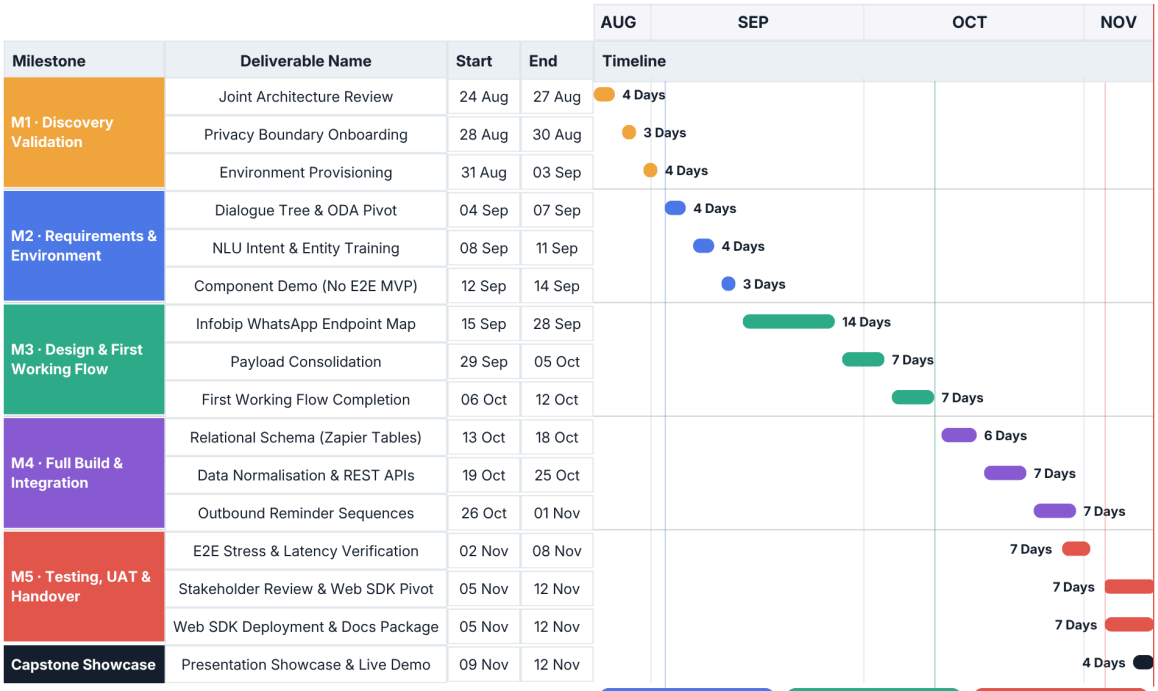
Planned Project Gantt Chart



Actual Progress Gantt Chart

Group 14 — UWC Health Clinic Patient Intake Bot (IFS325)

Section 12: Project Gantt Chart — Actual Progress & Execution Timeline



Actual execution window: 24 Aug — 12 Nov · 14 deliverables across 5 sprint milestones

References

De Vaal, S.J., 2015. Late booking at the Michael Mapongwana antenatal clinic, Khayelitsha–understanding the reasons (Doctoral dissertation, Stellenbosch: University of Stellenbosch).